

Paraphraser Fingerprinting: Can We Identify the Rewriter from the Text Alone?

CS6120 Project Report
Northeastern University
Seattle, WA, USA

Abstract

Different language models leave distinct stylistic traces—“fingerprints”—when paraphrasing text. We investigate whether these fingerprints are detectable by designing an adversarial evaluation framework in which an *Auditor* generates challenging source texts, four diverse paraphraser rewrite them, and a *Detective* attempts to identify which two outputs originate from the same paraphraser. Across 10 independent runs totaling 150 scored trials, the Detective achieves a mean accuracy of 60.7% (95% CI: [52.1%, 69.2%]), significantly above the 10% random baseline ($t(9) = 13.41, p < 0.001$). Detection rates vary markedly across paraphraser: GPT-5-Nano is identified with 100% accuracy, while Llama-3.1-8B proves hardest to detect at 25%. These results confirm that paraphraser fingerprinting is feasible and that different model architectures produce distinguishable rewriting patterns.

1 Problem Description

Large language models (LLMs) are increasingly used to paraphrase, rewrite, and polish text. A natural question arises: *do different models leave distinct traces when they paraphrase?* If so, it may be possible to identify *which* model produced a given rewrite—analogueous to identifying the “wood grain” of an AI model.

This question has practical implications for AI-generated text detection, academic integrity, intellectual property, and understanding model behavior. Prior work on AI text detection has focused on distinguishing human-written from machine-generated text [1], but the finer-grained problem of *paraphraser attribution*—identifying which specific model performed a rewrite—remains underexplored.

We formalize this as a **paraphraser fingerprinting** task: given a set of paraphrased outputs from multiple models (where one model contributes two outputs and the rest contribute one each), can a detector identify the two outputs that share the same origin? This framing isolates stylistic fingerprints from content, since all outputs derive from the same source text.

Our research question is:

RQ: Paraphraser Fingerprints. Do different paraphrasing models leave distinct, detectable traces in their outputs? Can we identify the paraphraser from the text alone?

2 Dataset

We do not use a pre-existing dataset. Instead, we generate all experimental data dynamically through our adversarial evaluation framework. This design choice is deliberate for several reasons:

- (1) **No suitable benchmark exists.** Existing paraphrase datasets (e.g., MRPC, QQP) contain human-written paraphrase pairs, not outputs from multiple competing paraphraser applied to the same source.

- (2) **Source text diversity matters.** The detectability of fingerprints depends heavily on the source text. Our framework generates source texts specifically designed to elicit distinguishable behaviors from different paraphraser.
- (3) **Adversarial refinement.** Through an iterative Auditor–Detective feedback loop, source texts are progressively refined to maximize fingerprint visibility, creating a more challenging and informative evaluation than static benchmarks would allow.

Each experimental run produces 20 trials. Only the initial trial uses a human-crafted source text containing deliberate linguistic traps (non-standard grammar, intentional repetition, temporal disorder, counting exercises). All subsequent trials use source texts generated by the Auditor LLM using Automatic Prompt Engineering (APE)-style iterative refinement based on past Detective feedback.

3 Experimentation and Results

3.1 Experimental Setup

Framework Architecture. Our system consists of three roles:

- **Auditor** (Claude Sonnet 4.6): Generates source texts designed to elicit distinguishable paraphrasing behaviors. Receives feedback from the Detective’s performance to iteratively refine its strategy.
- **Detective** (Claude Sonnet 4.6): Analyzes a set of 5 paraphrased outputs (from 4 paraphraser, one contributing two outputs) and identifies which two share the same origin. Operates statelessly in most configurations.
- **Paraphraser** (4 models):
 - Llama-3.1-8B-Instruct (via OpenRouter)
 - Gemini-2.5-Flash (via OpenRouter)
 - GPT-5-Nano (via OpenRouter)
 - DIPPER (T5-based, via HuggingFace Endpoint, $\text{lex} = 20, \text{order} = 40$)

Trial Protocol. Each trial proceeds as follows:

- (1) The Auditor generates (or selects) a source text.
- (2) One paraphraser is randomly chosen to produce *two* outputs; the remaining three each produce one output, yielding 5 total outputs.
- (3) The outputs are shuffled randomly.
- (4) The Detective examines all 5 outputs and predicts which pair originated from the same paraphraser.
- (5) A results block (correct/incorrect, predicted vs. actual indices) is fed back to the Auditor.

Configurations. We ran 10 independent experiments, each consisting of 20 trials with 5 warmup trials (not scored). Two configurations were tested: (1) a separate Detective with no memory across

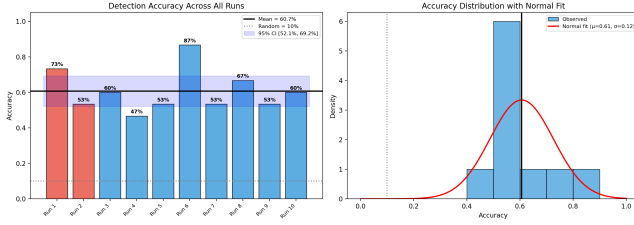


Figure 1: Detection accuracy across all 10 runs. Left: per-run accuracy with mean and 95% CI. Right: accuracy distribution with normal fit. The mean accuracy of 60.7% is significantly above the 10% random baseline ($t(9) = 13.41$, $p < 0.001$).

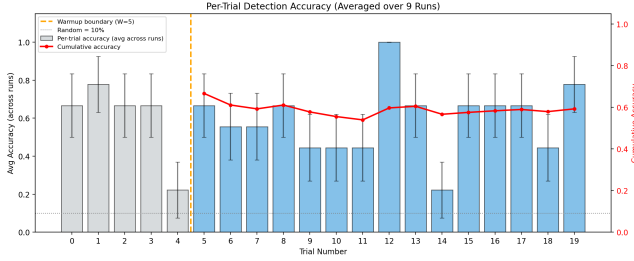


Figure 2: Per-trial detection accuracy averaged over 9 runs with full data. Blue bars show average accuracy at each trial position; the red curve shows cumulative accuracy from trial 5 onward. Error bars indicate standard error.

trials ($n = 2$ runs), and (2) the Auditor serving as Detective with access to past outputs and results ($n = 8$ runs).

Baseline. Random guessing yields $\frac{1}{\binom{5}{2}} = \frac{1}{10} = 10\%$ accuracy, since the Detective must select 2 of 5 outputs.

3.2 Aggregate Accuracy

Across 10 independent runs (150 total scored trials), the system achieves a mean detection accuracy of **60.7%** ($SD = 0.12$), with a 95% confidence interval of [52.1%, 69.2%]. A one-sample t -test against the random baseline confirms statistical significance: $t(9) = 13.41$, $p < 0.001$ (Figure 1).

Individual run accuracies range from 46.7% to 86.7%, reflecting the inherent stochasticity of random paraphraser duplication assignment and LLM output variability.

3.3 Per-Trial Learning Curve

Figure 2 shows the per-trial accuracy averaged across runs. The first 5 trials serve as warmup. The cumulative accuracy curve (red) shows the running average from trial 5 onward, revealing the Detective’s performance trajectory as the Auditor iteratively refines its source text generation strategy.

3.4 Per-Paraphraser Detection Rates

Detection rates vary dramatically across paraphraser (Figure 3):

- **GPT-5-Nano:** 100% (36/36)—always detected when duplicated.

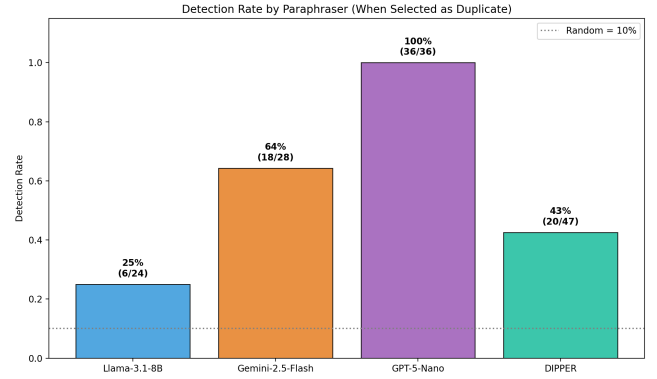


Figure 3: Detection rate for each paraphraser when randomly selected as the duplicate source. GPT-5-Nano is detected with 100% accuracy, while Llama-3.1-8B is hardest to detect at 25%.

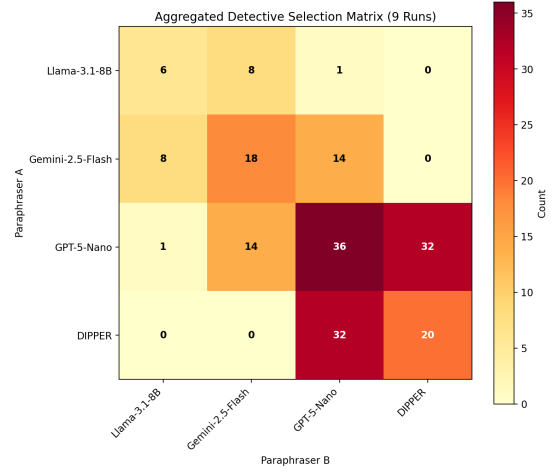


Figure 4: Aggregated confusion matrix showing how often the Detective paired each combination of paraphraser. Diagonal entries represent correct identifications. The strong GPT-5-Nano-DIPPER off-diagonal (32) indicates the Detective frequently confuses these two.

- **Gemini-2.5-Flash:** 64.3% (18/28).
- **DIPPER:** 42.6% (20/47).
- **Llama-3.1-8B:** 25.0% (6/24)—hardest to detect.

GPT-5-Nano’s perfect detection rate suggests it produces highly consistent, distinctive paraphrases—its “fingerprint” is strong and stable. Llama-3.1-8B’s low detection rate indicates high output variability, making its two outputs less recognizably similar to each other.

3.5 Confusion Analysis

The aggregated confusion matrix (Figure 4) reveals systematic patterns in the Detective’s errors:

- **GPT-5-Nano ↔ DIPPER** (off-diagonal: 32): The most common confusion pair. Despite their architectural differences

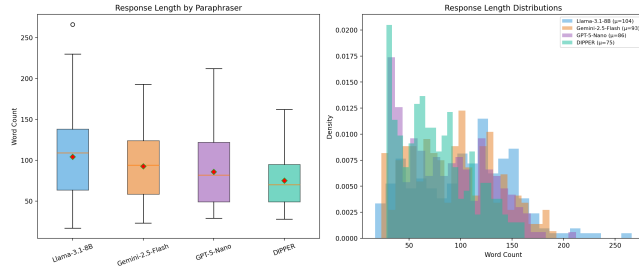


Figure 5: Response length (word count) distributions per paraphraser. Left: box plots. Right: overlaid histograms. Llama-3.1-8B produces the longest outputs ($\mu = 104$), while DIPPER produces the shortest ($\mu = 75$).

Table 1: Response length statistics (word count) per paraphraser.

Paraphraser	Mean	Median	SD	n
Llama-3.1-8B	104	109	47	210
Gemini-2.5-Flash	93	94	41	216
GPT-5-Nano	86	82	41	228
DIPPER	75	70	33	246

(LLM vs. T5-based), the Detective frequently groups their outputs together.

- **Gemini-2.5-Flash** $\leftrightarrow$ **GPT-5-Nano** (14): Moderate confusion between these two LLM-based paraphrasers.
- **Llama-3.1-8B** $\leftrightarrow$ **Gemini-2.5-Flash** (8): Some confusion between these models.
- **DIPPER** $\leftrightarrow$ **Llama-3.1-8B** (0): Never confused, suggesting maximally distinct fingerprints.

3.6 Response Length as a Fingerprint Feature

Response length is a salient fingerprint dimension (Figure 5). Table 1 summarizes the statistics.

DIPPER, as a T5-based model, produces the most conservative (shortest) paraphrases, while Llama-3.1-8B tends toward verbose, elaborative rewrites. Interestingly, despite GPT-5-Nano having moderate length, its fingerprint is the strongest—suggesting that length alone is insufficient and that deeper stylistic features (word choice, sentence structure, formatting patterns) play a critical role.

3.7 Source Text Type Comparison

We compared detection accuracy between human-crafted source texts (trials 5–7, post-warmup) and Auditor-generated texts (trials 8–19). Both achieve identical accuracy of 59.3% (Figure 6). This result suggests that the Auditor’s APE-style iterative refinement is competitive with carefully hand-crafted linguistic traps, validating the automated source text generation approach.

3.8 Detective Rationale Analysis

Analysis of 135 Detective rationales reveals the features most relied upon for fingerprint identification (Figure 7): *structure* (64%), *style*

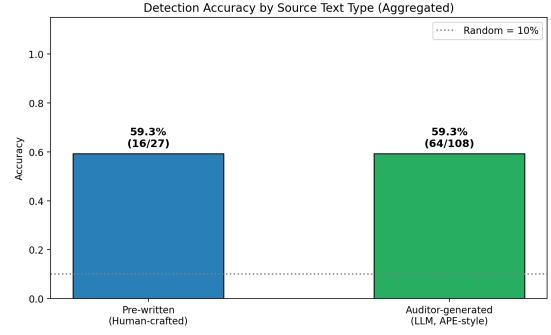


Figure 6: Detection accuracy by source text type. Both human-crafted and Auditor-generated texts yield identical accuracy (59.3%), suggesting the APE-style refinement matches human-designed linguistic traps.

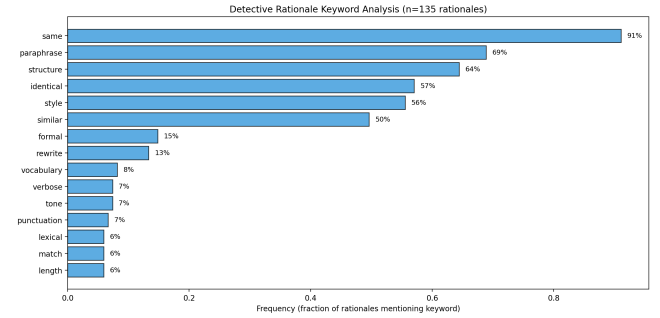


Figure 7: Keyword frequency analysis of Detective rationales ($n = 135$). Structural and stylistic features dominate the Detective’s reasoning.

(56%), *formal* (15%), *vocabulary* (8%), *tone* (7%), and *length* (6%). The dominance of structural and stylistic features over surface-level features like length confirms that the Detective performs genuine stylistic analysis rather than relying on trivial heuristics.

3.9 Auditor-as-Detective Configuration

We tested whether giving the Detective access to past outputs and results (“Auditor-as-Detective” configuration) improves performance. The separate Detective achieves 63.3% ($n = 2$) vs. 60.0% for Auditor-as-Detective ($n = 8$) (Figure 8). The lack of improvement suggests that within-trial stylistic comparison is more informative than cross-trial memory, or that the current memory mechanism does not effectively leverage historical information.

4 Plan for Next Steps

Based on our findings, we propose the following directions:

- (1) **Upgrade Detective model.** Replace Claude Sonnet 4.6 with a more capable model (e.g., Claude Opus, GPT-5) to test whether stronger reasoning improves fingerprint detection.
- (2) **Prompt optimization.** Systematically optimize the Detective’s prompt using techniques such as chain-of-thought

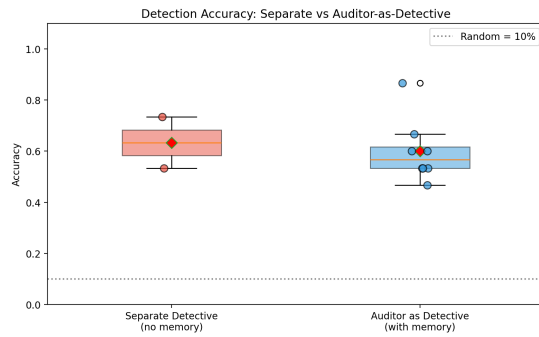


Figure 8: Comparison between separate Detective (stateless, $n = 2$) and Auditor-as-Detective (with memory, $n = 8$). No significant difference is observed.

prompting, few-shot examples of successful identifications, or structured comparison templates.

- (3) **Expand paraphraser pool.** Add more paraphraser (e.g., Mistral, Command-R, Qwen) to test whether fingerprinting scales to larger model sets and whether models from the same family (e.g., Llama-3.1-8B vs. Llama-3.1-70B) are distinguishable.
- (4) **Feature-based detection.** Complement the LLM-based Detective with traditional NLP features (n-gram distributions, syntactic parse patterns, vocabulary richness metrics) to build an interpretable fingerprint classifier.
- (5) **Cross-domain evaluation.** Test whether fingerprints persist across different text domains (scientific, literary, conversational) and source text lengths.
- (6) **Robustness testing.** Evaluate whether fingerprints survive post-processing steps such as grammar correction, back-translation, or additional paraphrasing.

5 Conclusion

We have demonstrated that different paraphrasing models leave detectable fingerprints in their outputs. Our adversarial evaluation framework, combining an iterative Auditor with a stateless Detective, achieves 60.7% mean accuracy across 150 scored trials—six times the random baseline of 10% ($p < 0.001$).

Key findings include:

- Fingerprint strength varies dramatically across models: GPT-5-Nano is always detectable (100%), while Llama-3.1-8B is rarely detected (25%).
- The Detective relies primarily on structural and stylistic features rather than surface-level heuristics like output length.
- Automated source text generation (APE-style) matches the effectiveness of human-crafted linguistic traps.
- Memory-augmented detection does not outperform stateless detection in our current framework.

These results suggest that the “wood grain” of AI models is real and measurable, opening avenues for paraphraser attribution, AI text provenance, and deeper understanding of how different architectures shape language generation.

References

- [1] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. 2023. DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature. In *Proceedings of ICML 2023*.